

Software Architecture & Design

Final Report

By Group 1

Contents

1. Group Members	
2. Code of Conduct	
3. Introduction and Project Details	
4. Component Specification Diagram	
5. Sprint 1 Architecture	
i. Component Specification Diagram	
6. Sprint 2 Architecture	
i. Business Concept Model	
ii. Business Type Model	
iii. Use Case Diagram	
iv. Business Interfaces	
v. System Interfaces	
vi. Initial System Architecture	
7. Sprint 3 Architecture	
i. Compound Components	
ii. Applied Domain Independent Styles	
iii. Applied SOA Principles	
8. Test Cases	
9. Meeting Records	
10. GitHub Links	

Group Members

Amel Soumia Bouras, A00018158 *Project Owner*

Lyba Ahmad, A00012095 *Scrum Master*

Jochebed Nsiah Saah, NSI22527318 *Team Member*

Rania Malik, A00011818 *Team Member*

Code of Conduct

This code of conduct outlines the basic rules and requirements we will follow during this project to ensure the project runs smoothly and everyone is involved.

Teamwork and Respect

All team members are expected to collaborate respectfully and professionally throughout the project. Everyone should have an equal opportunity to share ideas and opinions. Harassment or disrespect of any kind will not be tolerated. If you experience or witness inappropriate behaviour, speak up immediately. Workloads must be distributed fairly among all members.

Accountability

Each team member is responsible for completing their assigned tasks on time and contributing to the overall success of the group. If you are struggling or falling behind, inform the team as early as possible so support can be provided.

Communication

All project-related communication will take place in the WhatsApp group chat. Members are expected to stay up to date with messages and respond in a timely manner. In addition, the team will hold weekly meetings to review progress, discuss upcoming tasks, and ensure everyone is aligned. These meetings will be kept brief and focused.

Conflict Resolution

Any conflicts should be addressed respectfully between the individuals involved. If the issue cannot be resolved, the team will step in to assist and ensure a fair outcome.

Use of Artificial Intelligence

AI tools must be used in accordance with university guidelines. The team may use AI to support learning and understanding, but members must not rely on it without fully understanding the context or content produced.

Consequences for Non-Performance

If a team member fails to complete their responsibilities, the following staged consequences will apply:

- **Stage 1- Verbal Warning**
Issued if no progress is visible on GitHub within the first third of the allocated task timeframe.
- **Stage 2- Task Reassignment**
If no progress or minimal progress (10% or less) is shown on GitHub, the task will be reassigned to another team member. The original assignee must buy a drink for the person taking over the task.
- **Stage 3- Escalation**
Continued lack of contribution will result in the issue being reported to the lecturer.

Introduction & Project Details

This project includes the design and implementation of a 'Share Price Comparison Webpage', where users can select up to two companies to view and compare share prices over a period of time, ranging up to two years. The webpage also supports offline functionality of the graph as stock data is persisted on the system.

The following points go into more detail about the functionality of the webpage:

- The system fetches daily stock data for a company, from an API, upon a user's request. The data is then saved locally in a database so that the user can access it offline.
- Users select a start and end date, limited to a range of the current date up to two years prior.
- The selected stocks appear on a graph where the y-axis contains price and the x-axis contains days in the specified time range.
- Stocks are labelled with their company's name on the graph, and have distinct colours to distinguish between them.

Functional Requirements:

1. The system shall allow the user to select a valid company by searching through a list of available companies or typing its name/share symbol.
2. The system shall allow the user to add up to two companies to the graph.
3. The system shall allow the user to select two different dates.
4. The system shall ensure that the selectable dates are limited to the current date and dates up to two years before.
5. The system shall check the database if share data is available for the user's request before retrieving data from the API.
6. The system shall allow the user to add a graph to their watchlist.
7. The system shall allow the user to view the watchlist (watchlist page) and select an item which will be displayed on the graph (home page).
8. The system shall request daily share data from the API if not found locally on the database.
9. The system shall persist any new requested data into the database.

10. The system shall display a line graph of the selected company/companies, showing price over time (days).
11. The system shall use distinct colours and labels to differentiate the companies on the graph.

Non-Functional Requirements:

1. Usability- the system shall provide a clean UI that is easy to navigate.
2. Robustness- the system shall validate all inputs and handle errors correctly to avoid crashing.
3. Maintainability- the system shall follow a layered architecture that is well designed and justified.
4. Modularity- the system shall be designed to allow components to be replaceable.

Technical Decisions

The API of choice for this project is *Financial Modeling Prep* as it is reliable, has many free features, provides both real-time and historical data for a number of major companies, and can be easily integrated into a database. Overall, it fits our project the best compared to other APIs, although limited, aligning with the requirements.

The use of a relational database rather than persisting JSON/CSV responses in a file ensures data integrity and ease of data filtering through structured querying in the code. For a small-scale local project, SQLite is more appropriate than a server database like MariaDB.

Component Specification Diagram

Synopsis of Software Architecture

Simple Architecture is a software design philosophy that prioritises clarity, maintainability, and separation of responsibilities over complexity [1]. It also provides a clean and organised foundation that meets current system requirements while remaining adaptable to future changes [1]. Thus, it aligns with Clean Architecture principles, reducing coupling and keeping business logic independent of external frameworks and infrastructure [2]. In this project, Simple Architecture supports the core functionality of retrieving, storing, and comparing share price data without over engineering. A simple layered structure is sufficient, enabling focus on correctness, clarity, and maintainability. [1], [2]

Core Architectural Principles

The following principles are based on software architecture and Clean Architecture concepts [1], [2]:

Separation of Concerns – Divides the system into Presentation, Service, Data, and Domain layers to limit the impact of change.

Single Responsibility Principle – Each component has one clear purpose: Share Service for business logic, Share Repository for data, and Api Client for external communication.

Layered Architecture – Dependencies flow from Presentation to Service to Data, with higher-level components relying on abstractions, improving modularity and testability.

Low Coupling and Abstraction – Components interact via interfaces, allowing changes, such as switching storage methods, without affecting other layers.

Why is suitable for this project?

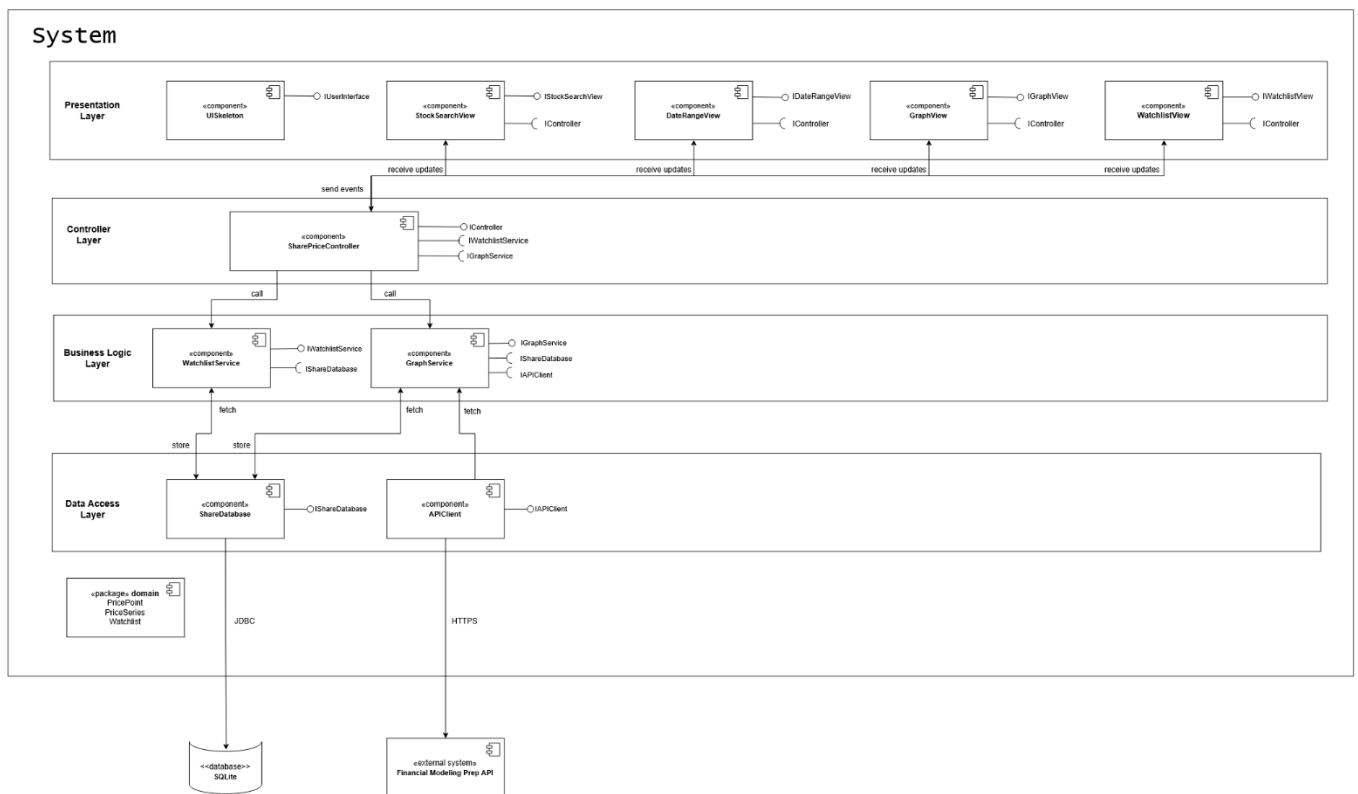
Simple Architecture fits the moderate scope of this project. The layered design supports collaboration, iterative development, and future enhancement, maintaining readability, testability, and extensibility without major restructuring. [1], [2]

References

[1] Lano, K. and Yassipour Tehrani, S. (2023) *Introduction to Software Architecture: Innovative Design using Clean Architecture and Model-Driven Engineering*.

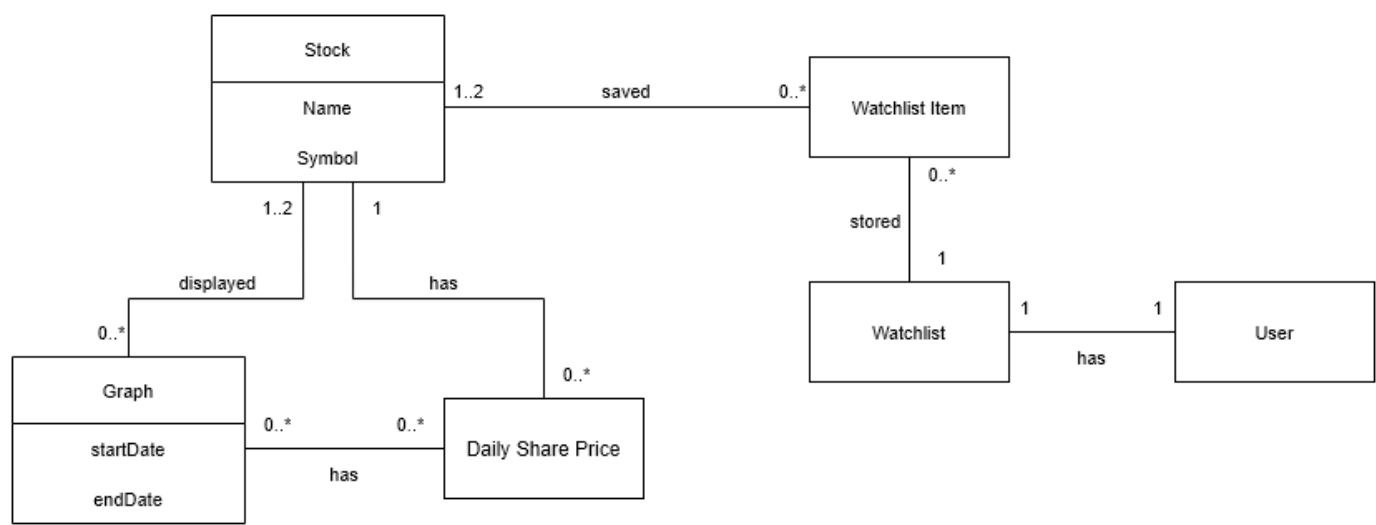
[2] Martin, R.C. (2018) *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Pearson Education.

Component Specification Diagram

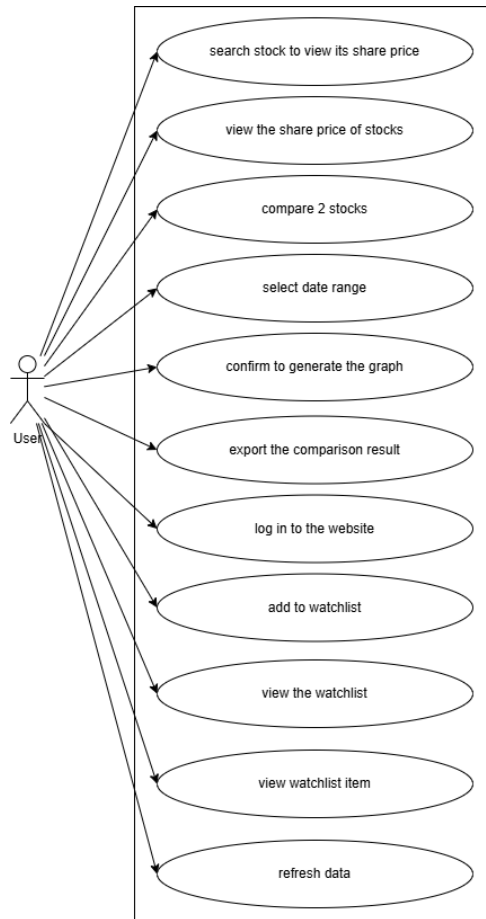


Sprint 2 Architecture

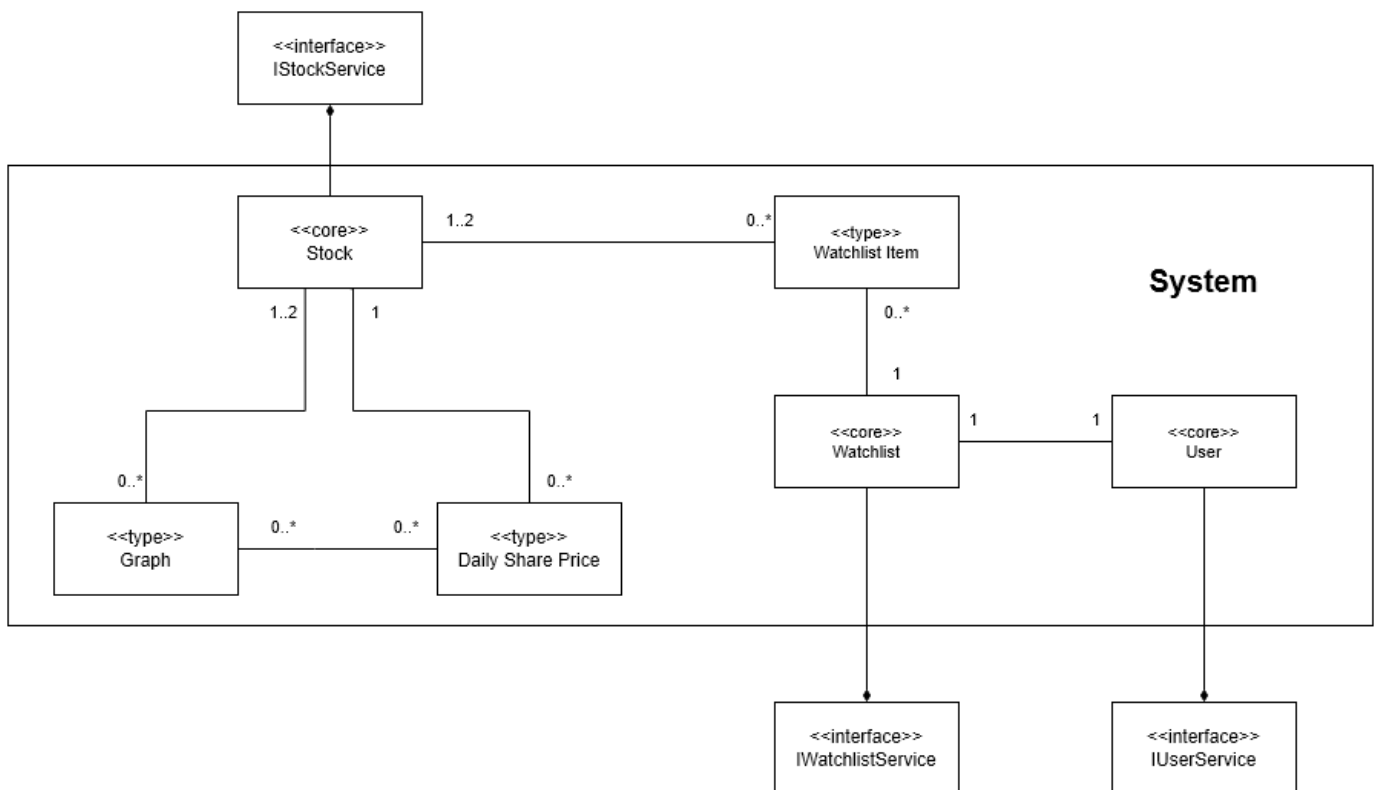
Business Concept Model



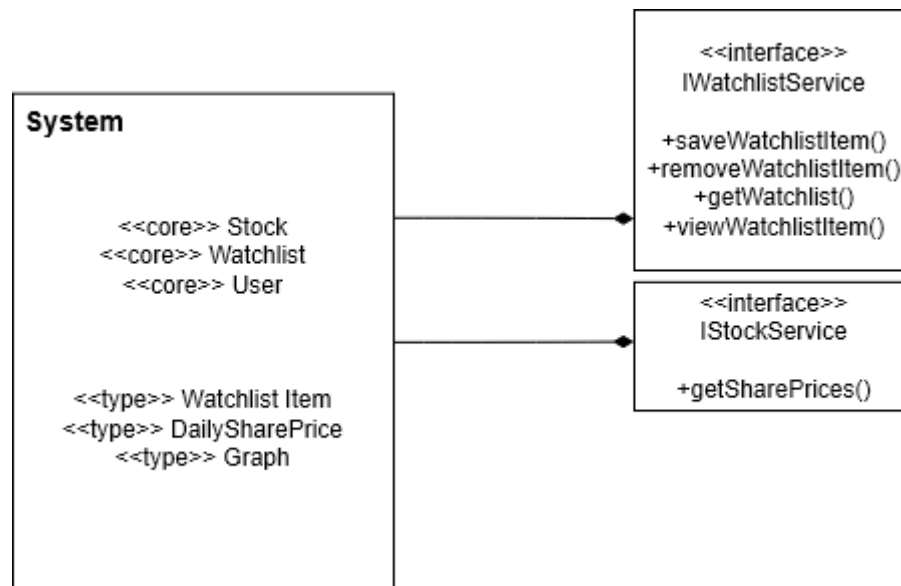
Use Case Diagram



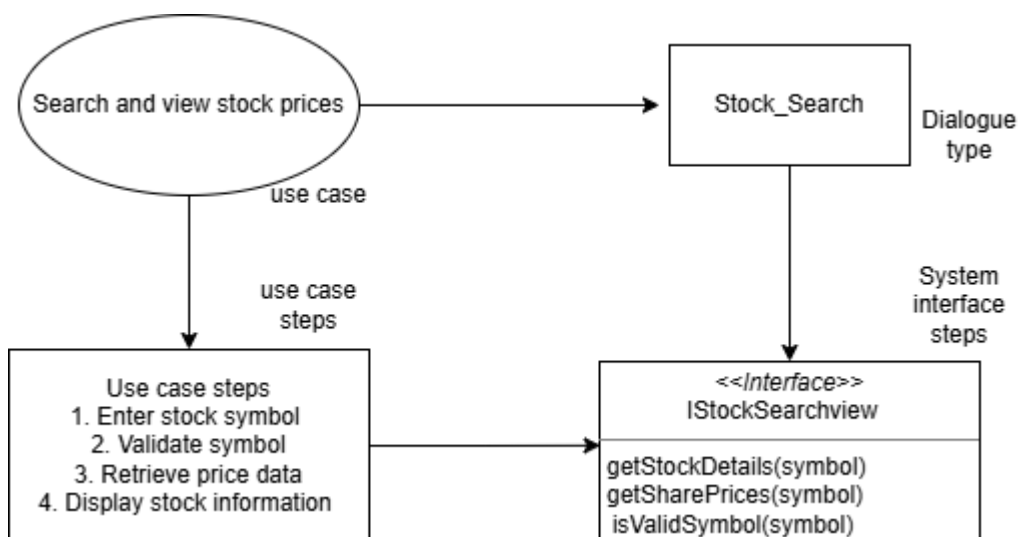
Business Type Model

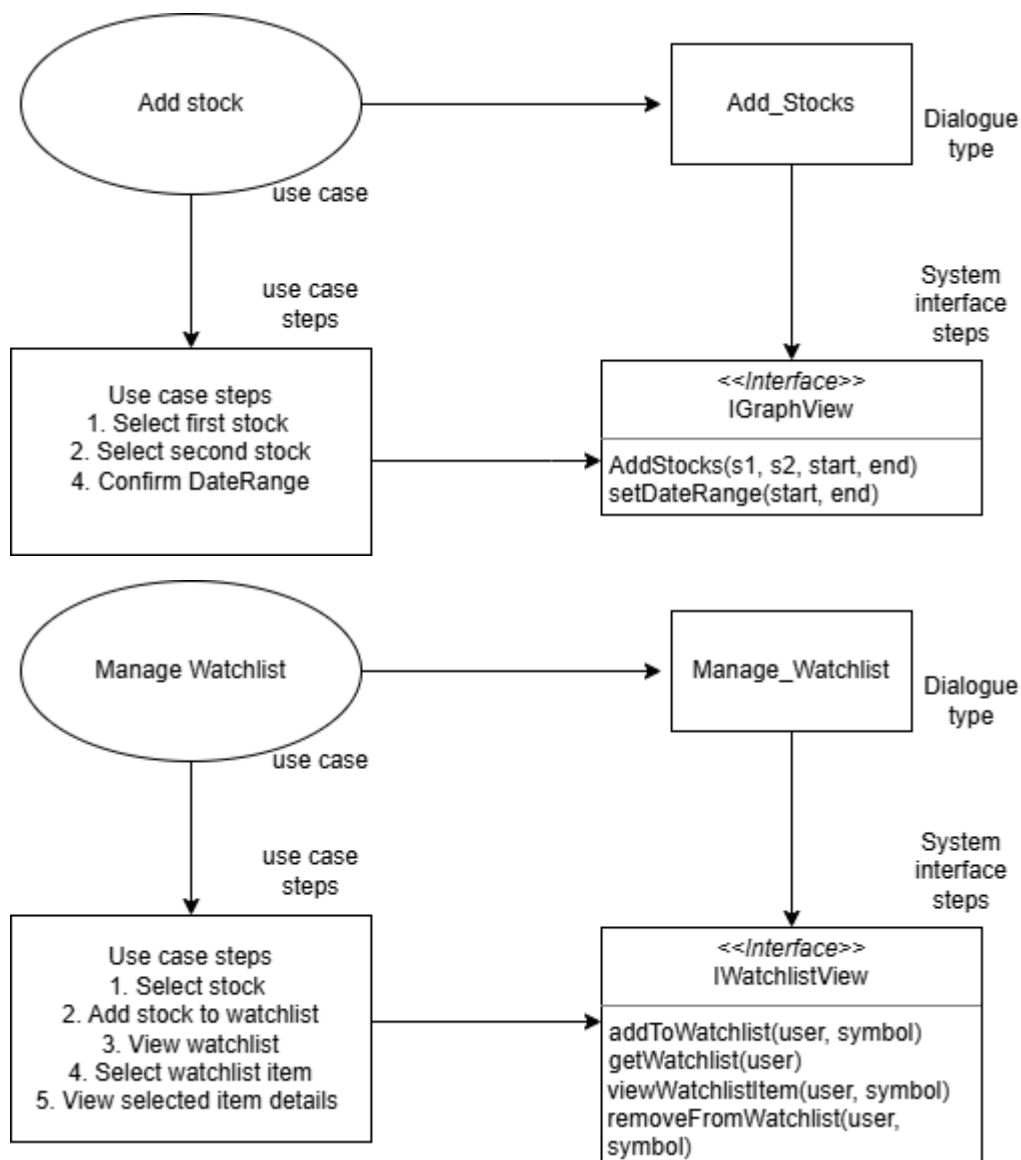


Business Interfaces

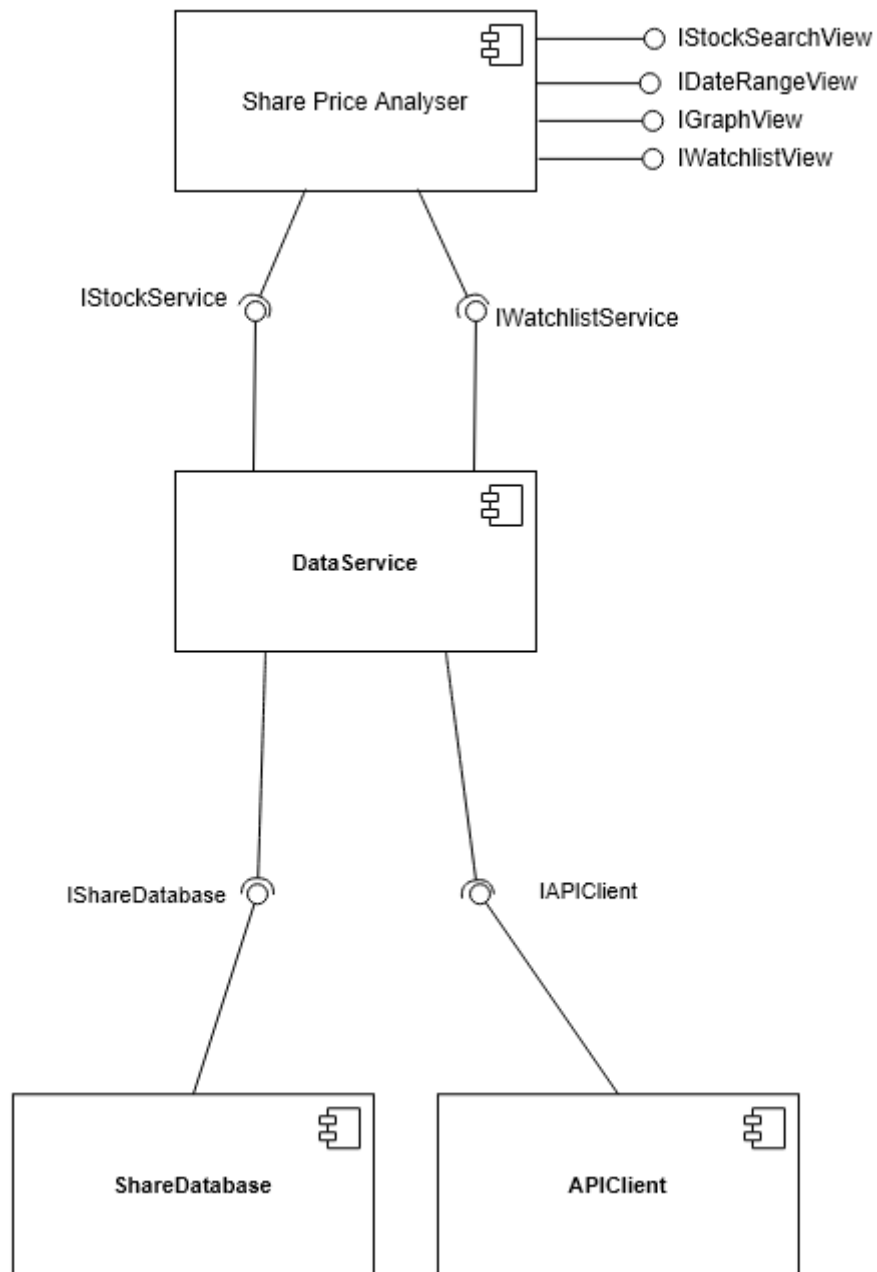


System Interfaces

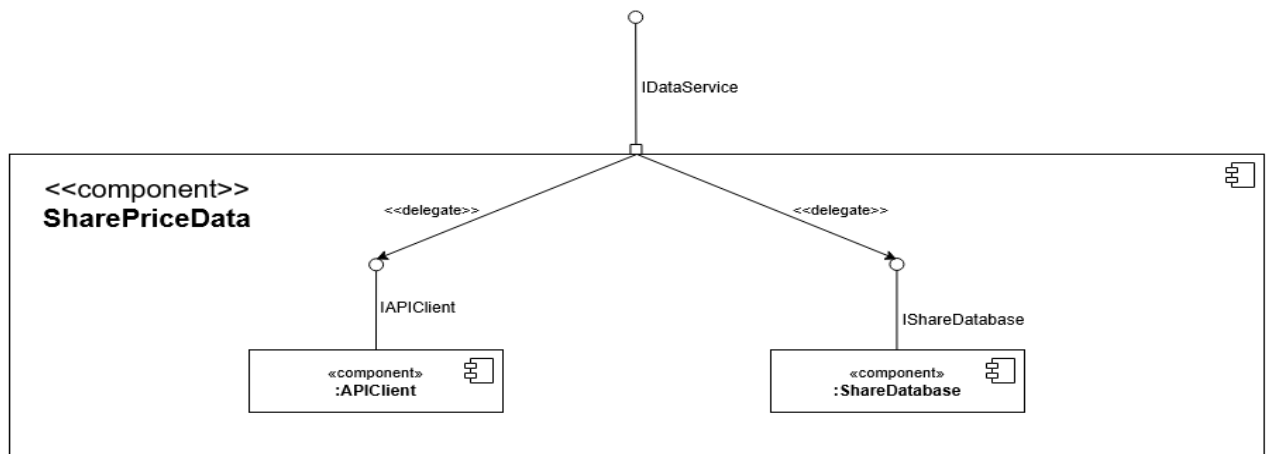
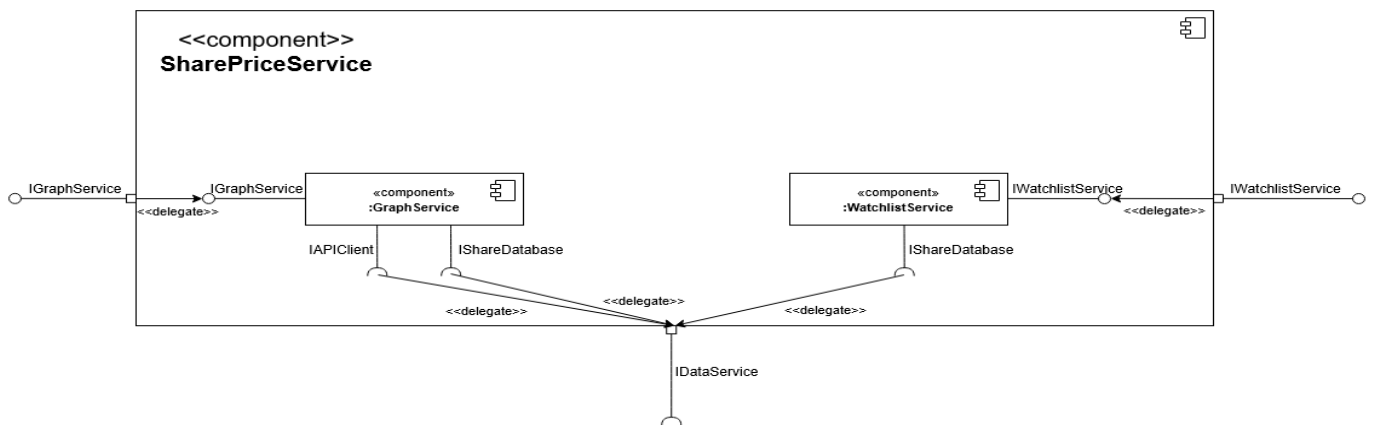
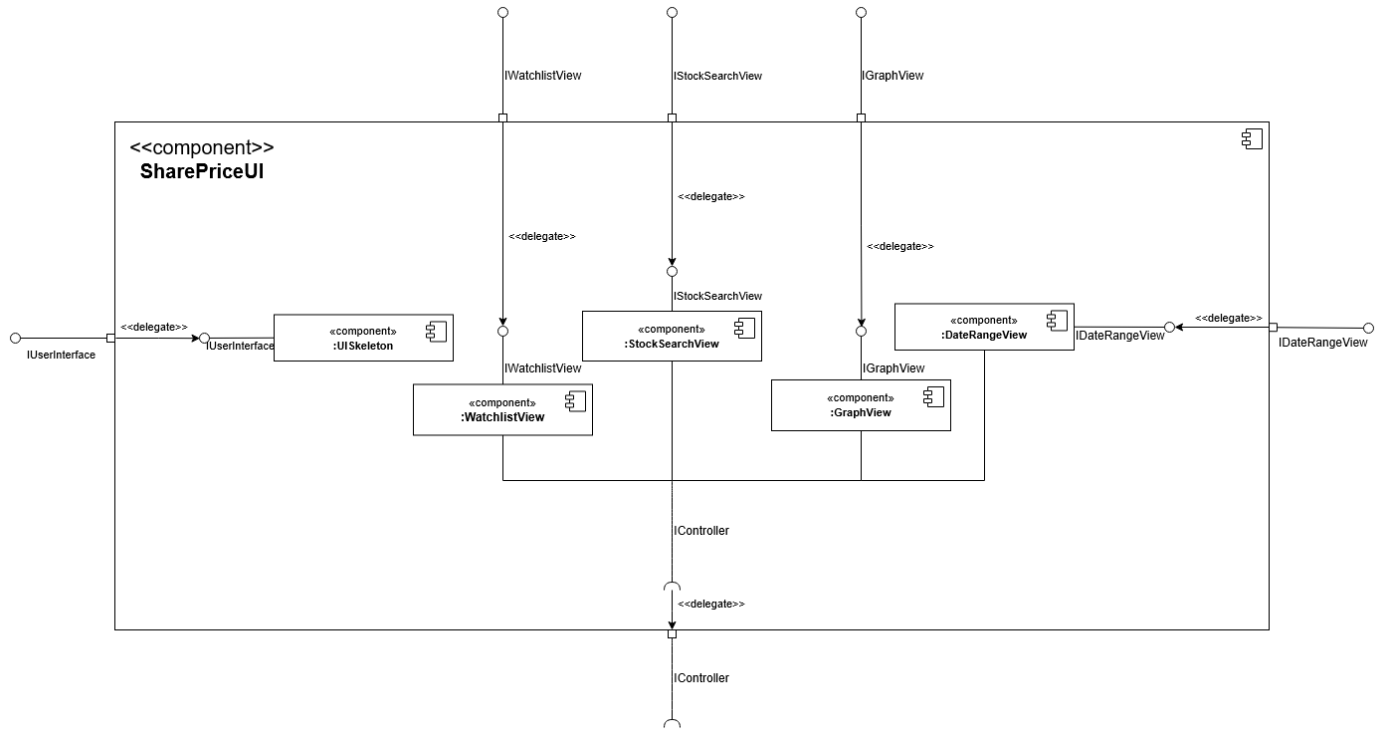




Initial System Architecture



Compound Components



Compound Component	Justification
SharePriceUI	All components encapsulated share a cohesive set of functions; they handle UI and user input, residing in the presentation layer. Most also require the same interface, IController, so having one compound component that interacts with SharePriceController, rather than multiple, is cleaner and improves modularity.
SharePriceService	Both components encapsulated handle business logic and share a required interface, IShareDatabase. It is therefore cleaner and more modular to combine them into one compound component that interacts with the data access layer (SharePriceData).
SharePriceData	Both components encapsulated handle and provide data from external sources (database, API). It is therefore cleaner and more modular to combine them into one compound component that provides one clear interface to the business logic layer (SharePriceService), maximising encapsulation.

Applied Domain Independent Styles

Our system was originally designed using a **layered** architecture, specifically **N-tier architecture** as it has 4 clear layers with separate responsibilities. **MVC** is also partially applied within these layers as the presentation layer acts as the 'view', SharePriceController as the 'controller', and the rest of the layers contribute to the 'model' responsibility. These were all applied to improve modularity which in turn creates a more reusable and maintainable system. This also helps separate the code to facilitate independent creation of the user interface and behind the scenes logic.

APIClient acts as an **Adapter** as it translates data taken from the external API to a readable format for the components in the business logic layer, completely encapsulating these functions within and only exposing the data through its provided interface. This further contributes to the clean architectural design as changing the API would have little affect on the system, making it replaceable.

The **Pipes and Filters** in GraphService contribute to the modularity of our system by encapsulating separate responsibilities into pipes and passing data through them using a filter to ensure data passes smoothly between the data access layer and the controller.

Applied SOA Principles

Although SOA principles weren't fully implemented, the use of our compound components and their exposed interfaces like 'IDataService' reflect the intended purpose; components of each layer expose the necessary data/functions while implementation details are hidden/encapsulated. Independent deployment wasn't necessary for our small project as it would overcomplicate the system, however, the benefits of SOA such as loose coupling, reusability and clear service boundaries, are reflected in our architecture.

Test Cases

Test Case	Action	Expected Result
Search stock	Type stock name or symbol in search bar, attempt empty search	Symbol suggestions appear (if provided by API), empty search returns nothing
Select stock	Select a symbol from the search suggestions	Correct stock symbol is returned, later displayed on the graph
Invalid date range	Exceeding the two year date range limit	Error message should show, nothing else returned or displayed
Invalid date input	Enter invalid date	Error message should show, nothing else returned or displayed
Select more than 2 stocks	Try selecting 3 stocks	Shouldn't be possible to search or add more than 2, or error message is shown and selection is ignored
Sumit all valid, expected inputs	Select a stock (or 2), and valid date ranges	Stocks displayed with correct information as a line graph
Add to watchlist	Click add to watchlist button	Watchlist item appears in watchlist page
View watchlist	Click on watchlist	Watchlist appears with all added items
Select from watchlist	Click a watchlist item	Correct graph/ item is loaded on the home page

Meeting Records

The records below highlight key group meetings made via WhatsApp call; other meetings include undocumented in-person and text discussions which occurred regularly during each sprint to ensure everyone was on track and on the same page.

Meeting 1

Date and Time	15-02-2026 12:00 – 12:55
Meeting Goal	• Agree on Code of Conduct • Assign roles in the team • Discuss and distribute tasks amongst members
Attendees	Amel, Lyba, Rania, Jochebed
Roundtable Updates	First team meeting so no new updates.
Discussion points	• Code of conduct • Component specification diagram • Researching architecture aspects • Deciding SCRUM Master and Product Owner • Product backlog and requirements
Actions	Code of Conduct – <i>Rania</i> Component diagram – <i>Lyba</i> Product backlog and requirements – <i>Amel</i> Research architectural designs – <i>Jochebed</i>

Meeting 2

Date and Time	17-02-2026 21:15 – 22:45
Meeting Goal	• Verify and finalise Component Specification Diagram • Distribute components amongst members to code
Attendees	Amel, Lyba, Rania, Jochebed
Roundtable Updates	Amel completed the requirements, Jochebed submitted initial research which Lyba used to draft a Component Specification diagram. Rania completed the Code of Conduct.
Discussion points	• Components and structure of the Component Specification Diagram • Initial talks about how to approach code implementation
Actions	Code Business Logic Layer component – <i>Rania</i> Code Presentation Layer components – <i>Lyba</i> Code Data Access Layer components – <i>Amel</i> Code Controller Layer component – <i>Jochebed</i>

Meeting 3

Date and Time	24/02/26 20:30- 21:00
----------------------	--------------------------

Meeting Goal	• Discuss Sprint 2 • Divide and assign tasks
Attendees	Amel, Lyba, Rania, Jochebed
Roundtable Updates	No new updates, reflecting on sprint 1.
Discussion points	• Sprint 2 task explanation • Consideration of changes to the original component diagram using feedback • Assigning components to each member based on new architecture • Dividing diagrams across members
Actions	<i>Amel</i>- business concept model & initial system architecture <i>Lyba</i>- use case & business interfaces <i>Rania</i>- system & business interfaces (1/2) <i>Jochebed</i>- business type model Everyone works on same layer as last sprint, making sure to align with simple architecture and updated business concept.

Meeting 4

Date and Time	13/03/26 20:00- 21:00
Meeting Goal	• Update on progress • Discuss improvements and ensure design consistency • Discuss integration into code
Attendees	Amel, Lyba, Jochebed
Roundtable Updates	Most diagrams complete, except final architecture.

Discussion points	- Progress each member made in sprint 2 - Improvements to diagrams for consistency and accuracy - Changes made to design that impact code, what to update and integrate
Actions	Complete outstanding diagrams and use feedback to update diagrams before code integration. Additional components assigned evenly among members.

Meeting 5

Date and Time	15/04/26 11:00 – 11:15
Meeting Goal	- Plan Sprint 3 - Split tasks between members
Attendees	Amel, Lyba, Rania, Jochebed
Roundtable Updates	No updates, reflecting on Sprint 2.
Discussion points	- Updated architecture choices - Tasks to assign to each member
Actions	Members to work on the same components as Sprint 2, fully implementing them and launching the website. <i>Amel</i> - Report, including Sprint 3 diagrams <i>Lyba</i> - Presentation slides

GitHub Links

Repository:

<https://github.com/Joch1n/share-price-analyser>

Kanban Board:

<https://github.com/users/Joch1n/projects/1>